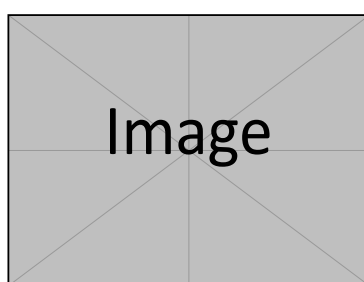


Crypto Viz

Plateforme d'Analyse Crypto en Temps Réel

Rapport Technique

Architecture et Choix Techniques



Projet T-DAT-901 Epitech

December 7, 2025

Contents

1	Résumé Exécutif	3
2	Architecture du Système	3
2.1	Architecture Générale	3
2.2	Composants Architecturaux	3
2.3	Flux de Données	4
3	Stack Technologique	5
3.1	Technologies Backend	5
3.2	Technologies Frontend	5
3.3	Infrastructure & DevOps	6
4	Décisions Techniques Clés	6
4.1	Architecture Orientée Événements avec Kafka	6
4.2	FastAPI pour l'API RESTful	6

4.3	Architecture des Calculateurs d'Analyse	7
4.4	D'etails des Calculs	7
4.4.1	VolatilityCalculator - Calcul de Volatilit'e	8
4.4.2	SharpeCalculator - Ratio de Sharpe	8
4.4.3	DrawdownCalculator - Calcul de Drawdown	9
4.4.4	CorrelationCalculator - Corr'elation entre Cryptomonnaies	10
4.4.5	PortfolioSimulator - Simulation de Portefeuille	11
5	Architecture du Frontend	12
5.1	Pages Principales	12
5.2	Architecture des Composants	12
6	Conception du Pipeline de Donn'ees	12
6.1	Phase d'Extraction	12
6.2	Phase de Chargement	12
6.3	Phase de Transformation	13
7	Conception de la Base de Donn'ees	13
7.1	Conception du Sch'ema	13
7.2	Consid'erations de Base de Donn'ees	13
8	Conception de l'API	14
8.1	Structure des Endpoints RESTful	14
8.2	Mod'eles de R'eponse	14
9	Consid'erations de S'ecurit'e	15
9.1	Mesures de S'ecurit'e Impl'ement'ees	15
9.2	Bonnes Pratiques de S'ecurit'e	15
10	Consid'erations de Scalabilit'e	16
10.1	Mise a` l'Echelle Horizontale	16
10.2	Optimisations de Performance	16
11	Architecture de D'eploiement	16
11.1	Configuration Docker Compose	16
11.2	Gestion de l'Environnement	16

12 Infrastructure de Test 17

12.1 Approche de Test 17

12.2 Outils de Test 17

13 Monitoring et Observabilit e 17

13.1 Monitoring Impl ment e 17

13.2 Impl mentation du Health Check 18

14 Exactitude Math matique et Bonnes Pratiques 18

14.1 Pr cision des Calculs Financiers 18

14.1.1 Volatilit  Bas e sur les Rendements 18

14.1.2 Agr gation Temporelle Adaptative 18

14.1.3 Ratio de Sharpe Honn te 19

14.1.4 Alignement Temporel pour la Corr lation 19

14.1.5 Pr cision du Drawdown avec Timestamps Exacts 20

14.2 Gestion des Fuseaux Horaires 20

14.3 Validation et Gestion d'Erreurs 20

15 D fis et Solutions 21

15.1 D fi 1 : Coh rence des Donn es en Temps R el 21

15.2 D fi 2 : Pr cision des M triques Financi res 21

15.3 D fi 3 : Performance de la Base de Donn es 21

16 Am liorations Futures 21

16.1 Am liorations Planifi es 21

16.2 Consid rations de Dette Technique 22

17 Conclusion 22**18 Annexes 22**

18.1 A. Structure du Projet 22

18.2 B. Variables d'Environnement Cl es 24

18.3 C. D pendances Python 25

18.4 D. D pendances Frontend 25

1 Résumé Exécutif

Crypto Viz est une plateforme complète d'analyse de cryptomonnaies en temps réel conçue pour fournir aux investisseurs et traders des informations exploitables. Le système combine la collecte de données continue, l'analyse avancée et la visualisation intuitive pour offrir une solution complète d'analyse du marché des cryptomonnaies.

Nom du projet	Crypto Viz - Plateforme d'Analyse Crypto en Temps Réel
Type	Application Web Full-Stack
Technologies principales	Python, Spark, FastAPI, PostgreSQL, Kafka, Next.js, Docker
Architecture	Microservices avec pipeline de données orientée événements
Déploiement	Conteneurs Docker avec orchestration Compose
Sources de données	CoinGecko API, CoinDesk API, Hyperliquid API

Table 1: Aperçu du projet

2 Architecture du Système

2.1 Architecture Générale

L'application suit une architecture microservices avec une séparation claire des responsabilités entre les couches d'ingestion de données, traitement, stockage et présentation.

2.2 Composants Architecturaux

Composant	Objectif
Couche d'Extraction	Récupère les données des APIs CoinGecko, CoinDesk et Hyperliquid
Courtier de Messages (Kafka)	Fait tampon et distribue les données entre les services
Couche de Chargement	Consomme les messages Kafka et les stocke dans la base de données
Couche de Transformation	Effectue l'analyse et crée les métriques dérivées
Couche API (FastAPI)	Expose les endpoints REST pour l'accès aux données
Couche Frontend (Next.js)	Fournit l'interface utilisateur et les visualisations
Base de Données (PostgreSQL)	Stocke les données historiques et en temps réel
Orchestration (Docker Compose)	Gère le cycle de vie des conteneurs et la mise en réseau

Table 2: Composants Architecturaux

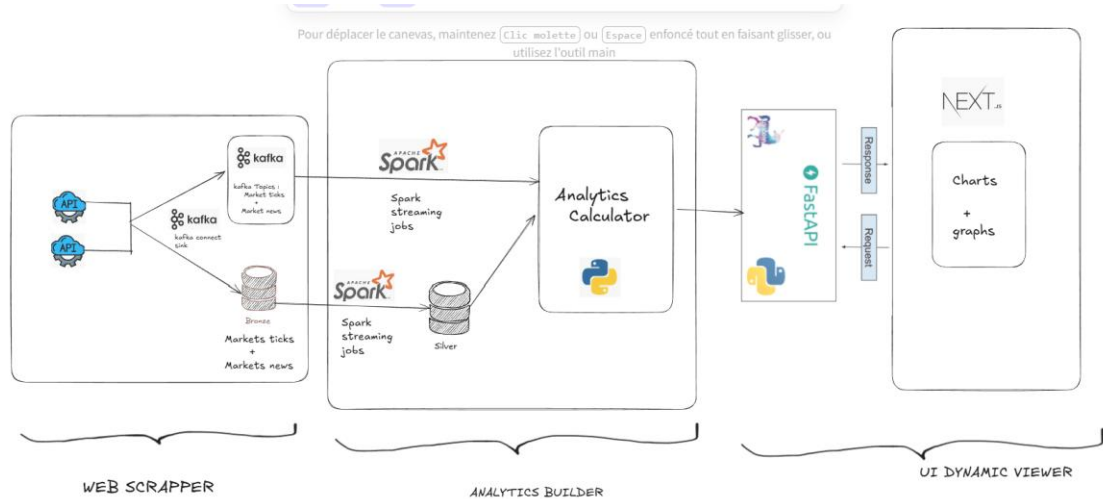


Figure 1: Diagramme d'Architecture du Syst`eme

2.3 Flux de Donnees

1. **Extraction** : CryptoDataProducer r`ecup`ere les donn`ees des APIs CoinGecko, CoinDesk et Hyperliquid a` intervalles configurables (par d`efaut : 180 secondes)
2. **Streaming** : Les donn`ees sont publi`ees sur les sujets Kafka pour traitement asynchrone
3. **Chargement** : CryptoDataConsumer lit depuis Kafka et stocke dans PostgreSQL
4. **Transformation** : Structuration de la data en s`eries temporelles et chargement dans le silver .
5. **Calculs** : Les calculateurs Python traitent les donn`ees pour g`en`erer les m`etriques
6. **Service** : FastAPI fournit les endpoints REST pour la consommation du frontend
7. **Visualisation**: Le frontend Next.js affiche les donn`ees via des graphiques interactifs

3 Stack Technologique

3.1 Technologies Backend

Technologie	Justification
Python 3.11	Langage principal pour le traitement des donn`ees et le d`eveloppement API
FastAPI	Framework web moderne et rapide avec documentation OpenAPI automatique

PostgreSQL	Base de données relationnelle avec option serverless pour l'extensibilité
Apache Kafka	Plateforme de streaming distribuée pour l'architecture orientée événements
kafka-python	Client Kafka pour Python utilisé par les producteurs et consommateurs
psycopg2	Adaptateur PostgreSQL pour Python avec support du pooling de connexions
pydantic	Validation des données et gestion des paramètres avec type hints Python

Table 3: Choix Technologiques Backend

3.2 Technologies Frontend

Technologie	Justification
Next.js 15	Framework React avec SSR, SSG et excellente expérience d' développeur
TypeScript	Typage statique pour une meilleure qualité de code et maintenabilité
Tailwind CSS	Framework CSS utilitaire pour développement UI rapide
Recharts	Bibliothèque de graphiques composable construite sur composants React
Fetch API	Client HTTP natif pour communication API
TanStack React Query	Gestion d'état serveur avec cache et synchronisation

Table 4: Choix Technologiques Frontend

3.3 Infrastructure & DevOps

Technologie	Justification
Docker	Conteneurisation pour développement et déploiement cohérents
Docker Compose	Orchestration pour développement local multi-conteneurs
PostgreSQL	PostgreSQL serverless pour gestion de base de données simplifiée
Git	Contrôle de version avec collaboration
python-dotenv	Gestion de la configuration d'environnement

Table 5: Choix Technologiques Infrastructure

4 Décisions Techniques Clés

4.1 Architecture Orientée Événements avec Kafka

Décision : Implémenter une architecture orientée événements utilisant Apache Kafka pour le pipeline de données.

Raisonnement :

- Découple les producteurs de données des consommateurs
- Permet le traitement des données en temps réel
- Fournit la persistance des messages et capacité de rejeu
- Mise à l'échelle horizontale pour augmenter le débit
- Supporte plusieurs consommateurs pour le même flux de données

Implémentation :

```
# Producteur (extract.py) producer = CryptoDataProducer(
kafka_broker=os.getenv("KAFKA_BROKERS", "kafka:9092"),
topic=os.getenv("KAFKA_TOPIC_MARKET", "market_data"), api_config=api_config,
symbols=COINS
)

# Consommateur (load.py) consumer =
CryptoDataConsumer( db_connection=db_conn,
kafka_broker=KAFKA_BROKERS,
topic=KAFKA_TOPIC_MARKET,
)
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14

4.2 FastAPI pour l'API RESTful

Décision : Utiliser FastAPI plutôt que Flask ou Django REST Framework.

Raisonnement :

- Validation de données intégrée avec Pydantic
 - Support asynchrone pour meilleures performances
 - Type hints Python modernes
 - Excellent benchmarks de performance
- Implémentation :**

```
# Structure API avec injection de dépendances app = FastAPI( title="API Crypto Viz",
description="Plateforme d'analyse crypto en temps réel", version="1.0.0", docs_url="/docs",
redoc_url="/redoc",
)

@router.get("/prices/latest", response_model=LatestPrices) async def get_latest_prices(db:
DatabaseConnection = Depends(get_db)):
    cursor = db.get_cursor()
    # ... opérations base de données
    return LatestPrices(data=prices, count=len(prices))
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14

4.3 Architecture des Calculateurs d'Analyse

Décision : Créer des classes de calculateurs modulaires pour différentes métriques financières.

Raisonnement :

- Séparation des responsabilités pour différents types d'analyse
- Composants réutilisables entre différents endpoints
- Facile à tester et maintenir
- Extensible avec de nouvelles métriques

Calculateur	Objectif
VolatilityCalculator	Mesure les fluctuations de prix et le risque
SharpeCalculator	Calcule les rendements ajustés au risque
DrawdownCalculator	Identifie les pertes maximales depuis les pics
CorrelationCalculator	Analyse les relations entre cryptomonnaies
PortfolioSimulator	Simule les stratégies d'investissement

Table 6 : Calculateurs d'Analyse

4.4 Détails des Calculs

Chaque calculateur implémente des formules mathématiques financières précises pour garantir l'exactitude des métriques.

4.4.1 VolatilityCalculator - Calcul de Volatilité

Le calculateur de volatilité mesure les fluctuations de prix en utilisant l'écart-type des rendements :

Formules mathématiques :

1. Calcul des rendements (en pourcentage) :

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}} \times 100 \quad (1)$$

2. Volatilité sur la période (écart-type des rendements) :

$$\sigma_{periode} = \sqrt{\frac{1}{N} \sum_{i=1}^N (R_i - \bar{R})^2} \quad (2)$$

3. Volatilité annualisée :

$$\sigma_{annuelle} = \sigma_{periode} \times \sqrt{n_{periodes/an}} \quad (3)$$

ou $n_{periodes/an} = 365$ (agrégation quotidienne) ou 365×24 (agrégation horaire)

4. Plage de prix :

$$Plage = Prix_{max} - Prix_{min} \quad (4)$$

Caractéristiques d'implémentation :

- Agrégation temporelle adaptative : quotidienne pour 14 jours, horaire pour >14 jours
- Calcul basé sur les rendements (approche financière standard)
- Agrégation des prix AVANT calcul des rendements (réduction du bruit)
- Métriques de volatilité sur période et annualisée

4.4.2 SharpeCalculator - Ratio de Sharpe

Le ratio de Sharpe mesure le rendement ajusté au risque en comparant le rendement excédentaire à la volatilité :

Formules mathématiques :

1. Rendement total (rendement réel sur la période) :

$$R_{total} = \frac{P_{fin} - P_{debut}}{P_{debut}} \times 100 \quad (5)$$

2. Rendement annualisé (hypothétique, si la tendance continue) :

$$R_{annuel} = \left[\left(\frac{P_{fin}}{P_{debut}} \right)^{\frac{365}{jours}} - 1 \right] \times 100 \quad (6)$$

3. Taux sans risque de période :

$$RFR_{periode} = RFR_{annuel} \times \frac{jours}{365} \quad (7)$$

4. Rendement excédentaire :

$$R_{exces} = R_{total} - (RFR_{periode} \times 100) \quad (8)$$

Volatilité de période (d'e-annualisée) :

$$\sigma_{periode} = \frac{\sigma_{annuelle}}{\sqrt{365/jours}} \quad (9)$$

6. Ratio de Sharpe (utilisant les rendements réels) :

$$Sharpe = \frac{R_{exces}}{\sigma_{periode}} \quad (10)$$

Classification du Ratio de Sharpe :

- Sharpe 3 : Exceptionnel
- Sharpe 2 : Excellent
- Sharpe 1 : Bon
- Sharpe 0 : Acceptable
- Sharpe < 0 : Médiocre

Innovation clé : L'implémentation utilise les rendements réels au lieu des rendements annualisés potentiellement trompeurs pour le calcul du Sharpe, évitant ainsi une confiance excessive dans les tendances à court terme.

4.4.3 DrawdownCalculator - Calcul de Drawdown

Le calculateur de drawdown identifie les pertes maximales depuis les pics de prix : **Formules mathématiques :**

1. Prix maximum courant :

$$P_{max}^{(t)} = \max(P_0, P_1, \dots, P_t) \quad (11)$$

2. Drawdown à l'instant t (depuis le pic courant) :

$$DD_t = \frac{P_t - P_{max}^{(t)}}{P_{max}^{(t)}} \times 100 \leq 0 \quad (12)$$

3. Drawdown maximal :

$$DD_{max} = \min (DD_0, DD_1, \dots, DD_n) \quad (13)$$

4. Valeur du drawdown maximal (en termes absolus) :

$$Valeur_{DD} = P_{pic} - P_{creux} \quad (14)$$

Drawdown actuel :

$$DD_{actuel} = \frac{P_{dernier} - P_{max}^{(n)}}{P_{max}^{(n)}} \times 100 \quad (15)$$

6. Pourcentage sous l'eau (% du temps sous le précédent sommet) :

$$\%_{sous-eau} = \frac{\sum_{i=1}^N I(P_i < 0.99 \times P_{max}^{(i)})}{N} \times 100 \quad (16)$$

Où I est la fonction indicatrice (seuil de 1%) 7.

Durée de déclin :

$$Duree = Timestamp_{creux} - Timestamp_{pic} \quad (17)$$

Détection de récupération :

$$Recupere = \begin{cases} Vrai & \text{si } P_{dernier} \geq 0.99 \times P_{max} \\ Faux & \text{sinon} \end{cases} \quad (18)$$

4.4.4 CorrelationCalculator - Corrélation entre Cryptomonnaies

Le calculateur de corrélation analyse les relations linéaires entre paires de cryptomonnaies :

Formules mathématiques :

1. Coefficient de corrélation de Pearson :

$$\rho(X, Y) = \frac{\sum_{i=1}^N (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum_{i=1}^N (X_i - \bar{X})^2 \times \sum_{i=1}^N (Y_i - \bar{Y})^2}} \quad (19)$$

où $-1 \leq \rho \leq +1$

2. Score de diversification :

$$Score = \max (0, \min (100, (1 - \rho_{moyen}) \times 100)) \quad (20)$$

Classification de la force de corrélation :

- $|\rho| \geq 0.9$: Très forte

- $|\rho| \geq 0.7$: Forte
- $|\rho| \geq 0.4$: Mod'érée
- $|\rho| \geq 0.2$: Faible
- $|\rho| < 0.2$: Très faible / Aucune

B'enefit de diversification :

- $\rho \geq 0.8$: M'ediocre (forte corrélation, faible diversification)
- $\rho \geq 0.5$: Mod'érée
- $\rho \geq 0.2$: Bon
- $\rho \geq -0.2$: Excellent
- $\rho < -0.2$: Exceptionnel (corrélation négative, couverture idéale)

Implémentation : Utilise des INNER JOIN sur les buckets temporels horaires pour garantir l'alignement des timestamps entre paires de cryptomonnaies.

4.4.5 PortfolioSimulator - Simulation de Portefeuille

Le simulateur de portefeuille permet de tester différentes stratégies d'investissement :

Formules mathématiques :

1. Quantité achetée :

$$Q = \frac{\text{Montant}_{\text{investissement}}}{P_{\text{achat}}} \quad (21)$$

2. Valeur actuelle :

$$V_{\text{valeur actuelle}} = Q \times P_{\text{actuel}} \quad (22)$$

Profit/Perte (P&L) :

$$P\&L = V_{\text{valeur actuelle}} - \text{Montant}_{\text{investissement}} \quad (23)$$

4. Pourcentage P&L (ROI) :

$$ROI = \frac{P\&L}{\text{Montant}_{\text{investissement}}} \times 100 \quad (24)$$

Fonctionnalités :

- **Simulation d'investissement unique** : Simuler un achat à une date spécifique, vente à une autre (ou au prix actuel)
- **Comparaison multi-crypto** : Comparer le même investissement sur toutes les cryptos, trié par P&L
- **Analyse du meilleur point d'entrée** : Trouver la date d'achat historique optimale pour un profit maximal

```

For each historical_price in lookback_period : quantity = investment /
    historical_price current_value = quantity * latest_price pnl
    = current_value - investment if pnl > best_pnl :
        Best_pnl = pnl best_entry = historical_price

```

Algorithme de recherche du meilleur point d'entrée :

1
2
3
4
5
6
7

5 Architecture du Frontend

5.1 Pages Principales

Le frontend Next.js fournit une interface utilisateur compl`ete avec les pages suivantes :

Page	Fonctionnalit`e
Dashboard	Affiche les prix actuels et les tendances du march`e en temps r`eel
Analytics	Pr`esente les m`etriques d`etaill`ees (volatilit`e, Sharpe, drawdown)
Correlation	Affiche la matrice de corr`elation entre les cryptomonnaies
Simulator	Permet de simuler les gains/pertes et les meilleurs points d'entr`ee

Table 7 : Pages du Frontend

5.2 Architecture des Composants

Le frontend utilise une architecture composant modulaire avec :

- **Composants UI** : Biblioth`eque compl`ete de composants r`eutilisables
- **Hooks Personnalis`es** : use-crypto-data, use-mobile pour logique m`etier
- **Gestion d'Etat** : React Query pour la synchronisation avec le backend
- **Styling** : Tailwind CSS avec th`emes personnalis`es

6 Conception du Pipeline de Donn`ees

6.1 Phase d'Extraction

Conception : Le producteur r`ecup`ere les donn`ees des APIs CoinGecko, CoinDesk et Hyperliquid `a intervalles r`eguliers.

Caract`eristiques :

- Intervalle de polling configurable (par d`efaut : 180 secondes)

- Support de multiples cryptomonnaies
- Gestion d'erreurs avec logique de retry
- Formatage structurée des données avant publication
- Support de multiples sources de données API

6.2 Phase de Chargement

Conception : Le consommateur lit depuis Kafka et charge dans PostgreSQL.

Caractéristiques :

- Pooling de connexions à la base de données
- Insertions par lots pour l'efficacité
- Validation des données avant insertion
- Initialisation et gestion du schéma

6.3 Phase de Transformation

```
# Calcul de volatilité en temps réel
calculator = VolatilityCalculator(get_db_config()) metrics =
calculator.calculate_volatility(coin_id, days=30)

# Calcul du ratio de Sharpe sharpe = SharpeCalculator(db_config) results =
sharpe.calculate_sharpe_ratio(coin_id, days=30)

# Calcul du drawdown maximal drawdown = DrawdownCalculator(db_config) max_dd =
drawdown.calculate_max_drawdown(coin_id, days=30)
```

Conception : Les calculateurs Python traite les données en temps réel.

1
2
3
4
5
6
7
8
9
10
11

7 Conception de la Base de Données

7.1 Conception du Schéma

```
-- Table principale de données de prix
CREATE TABLE crypto_prices (id SERIAL PRIMARY
    KEY, coin_id VARCHAR (50) NOT NULL,
    price_usd DECIMAL(20, 8), price_eur
    DECIMAL(20, 8), price_gbp DECIMAL(20, 8),
    change_24h DECIMAL(8, 4), market_cap
    BIGINT,
    timestamp TIMESTAMP WITH TIME ZONE NOT NULL, created_at TIMESTAMP WITH
    TIME ZONE DEFAULT NOW()
);

-- Indexes pour optimisation
CREATE INDEX idx_crypto_prices_coin_id ON crypto_prices(coin_id);
CREATE INDEX idx_crypto_prices_timestamp ON crypto_prices(timestamp);
CREATE INDEX idx_crypto_prices_coin_timestamp ON crypto_prices(coin_id, timestamp);
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17

7.2 Considérations de Base de Données

- **Indexation** : Créée sur les colonnes fréquemment interrogées (coin id, timestamp)
- **Partitionnement** : Considérer le partitionnement basé sur le temps pour les gros ensembles de données
- **Pooling de Connexions** : Implémenté dans la classe DatabaseConnection
- **Support SSL** : Requis pour les connexions PostgreSQL

8 Conception de l'API

8.1 Structure des Endpoints RESTful

L'API suit les principes REST avec des conventions de nommage cohérentes et des méthodes HTTP :

Méthode	Endpoint	Objectif
GET	/api/data/cryptos	Liste des cryptomonnaies suivies
GET	/api/data/prices	Données de prix historiques
GET	/api/data/prices/latest	Derniers prix pour toutes les cryptos
GET	/api/data/prices/{crypto id}	Données de prix pour une crypto spécifique
GET	/api/metrics/volatility	Métriques de volatilité pour toutes
GET	/api/metrics/volatility/{crypto id}	Volatilité d'une crypto spécifique
GET	/api/metrics/sharpe	Ratios Sharpe pour toutes
GET	/api/metrics/sharpe/{crypto id}	Ratio Sharpe d'une crypto spécifique
GET	/api/metrics/drawdown	Analyse de drawdown
GET	/api/metrics/drawdown/{crypto id}	Drawdown d'une crypto spécifique
GET	/api/metrics/correlation	Matrice de corrélation
POST	/api/simulate/pnl	Simulation de P&L
POST	/api/simulate/pnl/compare	Comparaison de plusieurs simulations
POST	/api/simulate/best-entry	Analyse du meilleur point d'entrée
GET	/api/simulate/best-entry/{crypto id}	Meilleur point pour une crypto
GET	/health	Vérification de santé du service
GET	/status	Etat global du système

Table 8: Endpoints API

8.2 Modèles de Réponse

Toutes les réponses API utilisent les modèles Pydantic pour une structure cohérente :

```
class PriceData(BaseModel):  
    id: int coin_id: str price_usd: float price_eur:  
    Optional[float] = None price_gbp: Optional[float]  
    = None timestamp: datetime
```

```
class VolatilityMetrics(BaseModel):  
    coin_id: str period_days: int
```

```
data_points: int mean_price: float  
annualized_volatility: float
```

```
class SharpeMetrics(BaseModel):  
    coin_id: str period_days: int data_points:  
    int total_return: float annualized_return:  
    float annualized_volatility: float  
    sharpe_ratio: float start_price: float  
    end_price: float
```

9 Considérations de Sécurité

9.1 Mesures de Sécurité Implémentées

- **Variables d'Environnement** : Les données sensibles stockées dans fichiers .env

- **SSL Base de Données** : Connexions SSL forcées à PostgreSQL
- **Configuration CORS** : Paramètres CORS adaptés (actuellement permissif pour développement)
- **Validation d'Entrée** : Les modèles Pydantic valident toutes les entrées API
- **Gestion d'Erreurs** : Messages d'erreur génériques en production

9.2 Bonnes Pratiques de Sécurité

```
# Connexion base de données avec SSL db_connection_string = f""" host={DB_HOST}
dbname={DB_NAME} user={DB_USER} password={DB_PASSWORD} port={DB_PORT}
sslmode=require
"""

# Configuration CORS pour développement
app.add_middleware( CORSMiddleware, allow_origins=[
    "http://localhost:3000",
    "http://localhost:5173",
    "http://localhost:8080",
    "*" # restreindre en production
], allow_credentials=True,
allow_methods=["*"],
allow_headers=["*"],
)
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19

10 Considérations de Scalabilité

10.1 Mise à l'Echelle Horizontale

L'architecture supporte la mise à l'échelle horizontale via :

- **Partitions Kafka** : Plusieurs instances de consommateurs peuvent traiter les données
- **API Stateless** : Les instances FastAPI peuvent être équilibrées en charge
- **Répliques de lecture BD** : Pour les charges de travail lecture-intensive
- **Microservices** : Mise à l'échelle indépendante des composants

10.2 Optimisations de Performance

- **Pooling de Connexions** : Réutilisation des connexions base de données
- **Couche Cache** : Potentiel d'intégration Redis
- **Agrégation de Données** : Métriques pré-calculées pour requêtes courantes
- **Traitement Asynchrone** : Opérations non-bloquantes

11 Architecture de Déploiement

11.1 Configuration Docker Compose

L'application utilise Docker Compose pour le développement local avec les services suivants :

Service	Configuration
kafka	Courtier de messages avec ZooKeeper
zookeeper	Gestion de cluster Kafka
extract	Conteneur producteur Python
load	Conteneur consommateur Python
transform	Structuration et construction des séries temporelles
api	Application FastAPI
frontend	Application Next.js
postgres	Base de données PostgreSQL (optionnel, Neon utilisée principalement)

Table 9: Services Docker

11.2 Gestion de l'Environnement

```
# Structure de configuration d'environnement src/
.env.example      # Variables d'environnement exemple api/
requirements.txt  # Dépendances Python API
```

1
2
3
4
5

```
... pipelines/  
extract/ utils/  
load/ utils/  
transform/ volatility_calculator.py  
          sharpe_calculator.py drawdown_calculator.py  
          correlation_calculator.py  
          portfolio_simulator.py  
utils/
```

6
7
8
9
10
11
12
13
14
15
16
17
18

12 Infrastructure de Test

12.1 Approche de Test

- **Tests Unitaires** : Fonctions individuelles de calculateurs et utilitaires
- **Tests d'Intégration** : Endpoints API avec base de données
- **Tests End-to-End** : Pipeline complet du données d'extraction à l'API
- **Tests de Performance** : Charge testing des endpoints API

12.2 Outils de Test

- pytest pour tests Python
- unittest pour tests unitaires basiques
- Postman/Newman pour tests API
- Docker pour isolation d'environnement de test
- Scripts de test dans le répertoire **scripts/** : testapi.py, test volatility.py, test sharpe.py, test drawdown.py, test correlation.py, test pnl simulator.py, test _neon data.py

13 Monitoring et Observabilité

13.1 Monitoring Implémentée

- **Endpoints de Santé** : /health et /status pour monitoring des services
- **Logging** : Logging structurée throughout l'application
- **Suivi d'Erreurs** : Gestion centralisée d'erreurs et reporting
- **Métriques** : Métriques de performance API et statistiques base de données

13.2 Implémentation du Health Check

```
@router.get("/health", response_model=HealthCheck) async def health_check(db:
DatabaseConnection = Depends(get_db)):
    try:
        cursor = db.get_cursor() cursor.execute("SELECT
1") cursor.fetchone() cursor.close() return
        HealthCheck( status="healthy",
timestamp=datetime.utcnow(),
database="connected",
)
    except Exception as e:
        raise HTTPException( status_code=503, detail=f"Service
unhealthy: {str(e)}",
)
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17

14 Exactitude Mathématique et Bonnes Pratiques

14.1 Précision des Calculs Financiers

L'implémentation des calculateurs respecte les standards de l'industrie financière avec plusieurs optimisations clés :

14.1.1 Volatilité Basée sur les Rendements

Approche correcte : Calcul de la volatilité à partir des rendements (variation relative)

```
# CORRECT : Utilisation des rendements return_pct = ((price - prev_price) /
prev_price) * 100 volatility = STDDEV_POP(return_pct)

# INCORRECT : Utilisation des prix bruts volatility = STDDEV_POP(price) # Ne tient pas
compte de l'
```

chelle

plutôt que des différences de prix absolues.

1
2
3
4
5
6

Justification : Les rendements sont normalisés et comparables entre actifs de prix différents (Bitcoin à \$100,000 vs Ethereum à \$4,000).

14.1.2 Agrégation Temporelle Adaptative

L'agrégation des prix avant le calcul des rendements réduit le bruit et améliore la précision :

```
WITH aggregated_prices AS (
  SELECT
    DATE_TRUNC('day', timestamp) as time_bucket,
    AVG(price_usd) as avg_price
  FROM crypto_prices
  WHERE period_days >= 14
  GROUP BY time_bucket
),
```

1
2
3
4
5
6
7
8

```

returns AS (
    SELECT
        ((avg_price - LAG(avg_price) OVER (ORDER BY time_bucket))
        / LAG(avg_price) OVER (ORDER BY time_bucket)) * 100 as return_pct
    FROM aggregated_prices
)
SELECT STDDEV_POP(return_pct) as volatility FROM returns;

```

9
10
11
12
13
14
15
16

Stratégie d'agrégation :

- Analyse à 14 jours : Buckets horaires ($n = 365 \times 24$)
- Analyse 14 jours : Buckets quotidiens ($n = 365$)
- Réduit le bruit des fluctuations intra-journalières
- Garantit suffisamment de points de données pour la significativité statistique

14.1.3 Ratio de Sharpe Honnête

Innovation clé : Utilisation des rendements réels au lieu des rendements annualisés pour le

```

# Calcul du rendement total réel total_return = ((end_price - start_price) / start_price) * 100

# Rendement annualisé (pour information seulement) annualized_return = (((end_price / start_price)
    ** (365 / days)) - 1) * 100

# Volatilité d'annualisé pour correspondre au rendement total period_volatility = annualized_volatility /
sqrt(365 / days)

# Sharpe ratio utilisant le rendement réel sharpe_ratio = (total_return - period_rfr * 100) / period_volatility

```

calcul du ratio de Sharpe.

1
2
3
4
5
6
7
8
9
10
11
12

Problème évité : L'annualisation de rendements courts (ex: +10% en 7 jours → +520% annualisé) crée des illusions de performance non durables. Notre implémentation prévient cette fausse confiance.

14.1.4 Alignement Temporel pour la Corrélation

L'utilisation d'INNER JOIN garantit que seules les paires de prix avec timestamps

```
WITH coin1_data AS (
    SELECT DATE_TRUNC('hour', timestamp) as time_bucket, AVG(price_usd) as price
    FROM crypto_prices WHERE coin_id = 'bitcoin' GROUP BY time_bucket
), coin2_data AS (
    SELECT DATE_TRUNC('hour', timestamp) as time_bucket, AVG(price_usd) as price
    FROM crypto_prices WHERE coin_id = 'ethereum' GROUP BY time_bucket
)
```

correspondants sont corrélées :

```
SELECT CORR(c1.price, c2.price) as correlation
FROM coin1_data c1
INNER JOIN coin2_data c2 ON c1.time_bucket = c2.time_bucket;
```

Avantage : Elimine les corrélations fausses dues à des données manquantes d'alignées.

14.1.5 Précision du Drawdown avec Timestamps Exacts

L'algorithme de drawdown conserve les timestamps exacts des pics et creux pour une traçabilité complète :

```

for i, row in enumerate(price_data): current_price = row['price_usd']
    timestamp = row['timestamp']

    if current_price > running_max: running_max = current_price peak_timestamp = timestamp
    drawdown = ((current_price - running_max) / running_max) * 100

    if drawdown < max_drawdown: max_drawdown = drawdown
        trough_price = current_price trough_timestamp =
        timestamp max_dd_peak_price = running_max
        max_dd_peak_timestamp = peak_timestamp

```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16

B'enefit : Permet l'analyse historique des 'ev'ènements de marché (ex: "le drawdown maximal de -45% s'est produit du 2024-11-15 au 2024-12-01").

14.2 Gestion des Fuseaux Horaires

```

def to_naive_utc(dt):
    """Convert datetime to naive UTC (no tzinfo)""" if dt.tzinfo is not None:
        return dt.astimezone(timezone.utc).replace(tzinfo=None)

    return dt

```

Toutes les dates sont converties en UTC naïf (sans tzinfo) pour coh'erence :

1
2
3
4
5

Probl'eme r'esolu : Incompatibilit'es entre datetime aware/naive lors des comparaisons et insertions base de donn'ees.

14.3 Validation et Gestion d'Erreurs

Chaque calculateur implémente une validation robuste :

- **Vérification de points de données suffisants** : Minimum 2 points pour rendements, 3+ pour statistiques significatives
- **Gestion des divisions par zéro** : Vérification que $\text{prev price} \neq 0$ avant calcul de rendement
- **Gestion des valeurs NULL** : Filtrage des enregistrements avec price usd NULL
- **Validation des périodes** : Contraintes sur les paramètres de période (ex: days ≥ 0)
- **Messages d'erreur descriptifs** : Logging détaillé pour debugging

15 Défis et Solutions

15.1 Défi 1 : Cohérence des Données en Temps Réel

Problème : Assurer la cohérence des données entre plusieurs services avec des fréquences de mise à jour différentes.

Solution : Opérations idempotentes implémentées et versioning de données dans la couche base de données.

15.2 Défi 2 : Précision des Métriques Financières

Problème : Calculer des métriques financières précises avec des intervalles de temps irréguliers.

Solution : Agrégation par buckets de temps basée sur la période d'analyse :

- Analyse 24h : Agrégation horaire
- Analyse 7 jours : Intervalles de 6 heures
- Analyse 30 jours : Intervalles quotidiens
- Analyse 90+ jours : Intervalles de 3 jours

15.3 Défi 3 : Performance de la Base de Données

Problème : Requêtes efficaces sur de gros ensembles de données historiques.

Solution : Implémentation de :

- Indexation stratégique sur coin id et timestamp
- Optimisation de requêtes avec EXPLAIN ANALYZE

- Agrégation de données au niveau API pour requêtes courantes

16 Améliorations Futures

16.1 Améliorations Planifiées

1. **WebSockets en Temps Réel** : Mises à jour de prix en direct sans polling
2. **Intégration Machine Learning** : Modèles de prédiction de prix
3. **Analytics de Portefeuille Avancée** : Calculs de théorie de portefeuille moderne
4. **Application Mobile** : App React Native compagnon
5. **Déploiement Cloud** : Orchestration Kubernetes sur AWS/GCP
6. **Sources de Données Supplémentaires** : Intégration de données spécifiques aux échanges

16.2 Considérations de Dette Technique

- **Migrations Base de Données** : Implémenter Alembic pour versioning du schéma
- **Versioning API** : Ajouter le versioning aux endpoints API
- **Implémentation Cache** : Ajouter Redis pour données fréquemment accédées
- **Stack de Monitoring** : Intégrer Prometheus et Grafana

17 Conclusion

Crypto Viz représente une implémentation robuste d'une plateforme d'analyse de cryptomonnaies en temps réel. L'architecture équilibre avec succès :

- **Performance** : Via traitement asynchrone et requêtes optimisées
- **Scalabilité** : Via architecture microservices et design orienté événements
- **Maintenabilité** : Avec structure de code modulaire et séparation claire des responsabilités
- **Extensibilité** : Via interfaces bien définies et pattern de calculateur
- **Utilisabilité** : Avec documentation API complète et frontend intuitif

Les choix techniques reflètent les bonnes pratiques modernes en développement fullstack, ingénierie de données et analyse financière. La plateforme fournit une base solide pour expansion future dans des outils d'analyse de cryptomonnaies plus avancés.

18 Annexes

18.1 A. Structure du Projet

t-dat-901-crypto-viz/ src/ api/

```
__init__.py
__main__.py main.py

database.py
models.py
requirements.txt
routers/
    __init__.py data.py
health.py
```

Application FastAPI
Gestion de connexion BD
Mod les Pydantic
D pendances Python
Modules de routes API
Routes de donn es
Routes de sant

1
2
3
4
5
6
7
8
9
10
11
12
13

```

metrics.py      # Routes de m triques simulation.py      # Routes de
simulation
pipelines/ extract/
    extract.py      # Producteur Kafka utils/
    extract_producer.py
load/
    load.py      # Consommateur Kafka utils/
    data_consumer.py
transform/ transform.py
    volatility_calculator.py
    sharpe_calculator.py
    drawdown_calculator.py
    correlation_calculator.py
    portfolio_simulator.py utils/
utils/ utils.py
test/ test.py
.env      # Configuration d'
environnement
crypto-dashboard/      # Frontend Next.js
app/
    page.tsx      # Dashboard principal layout.tsx analytics/
    page.tsx      # Page Analytics
correlation/
    page.tsx      # Page Corr lation
simulator/
    page.tsx      # Page Simulateur
components/
    analytics/ # Composants Analytics dashboard/ # Composants
    Dashboard simulator/ # Composants Simulateur correlation/ #
    Composants Corr lation shared/ # Composants partag s layout/ #
    Composants de layout providers.tsx
    ui/      # Composants UI
hooks/ use-crypto-data.ts use-
    mobile.ts use-toast.ts
lib/
    api.ts # Client API store.tsx utils.ts
package.json tsconfig.json
next.config.mjs
Dockerfile scripts/
test_api.py

```

16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37

38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70

```

test_volatility.py test_sharpe.py
test_drawdown.py test_correlation.py
test_pnl_simulator.py test_neon_data.py
verify_hyperliquid_data.py
seed_historical_data.py
seed_historical_data_v2.py
seed_historical_data_v3.py
clean_old_cryptos.py start_api.sh
start_full_stack.sh diagnose_drawdown.sh
test_fixed_metrics.sh
verify_metrics_corrections.sh
.config/ iac/dev/ docker-compose.yml      # Orchestration locale
.env.dev configs/ start_docker.sh        # Script de lancement
README.md
rapport.tex                               # Ce rapport

```

71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94

18.2 B. Variables d'Environnement Clés

Variable	Description
COINGECKO BASE	URL de base de l'API CoinGecko
COINDESK API KEY	Clé API CoinDesk
COINS	Liste des cryptomonnaies à suivre

VS	Devises de comparaison (par défaut : usd)
KAFKA BROKERS	Adresse du courtier Kafka
KAFKA TOPIC MARKET	Sujet Kafka pour données de marché
POLL INTERVAL SEC	Intervalle de polling en secondes (défaut : 180)
BRONZE DB HOST	Host PostgreSQL Neon
BRONZE DB NAME	Nom de la base de données Bronze
BRONZE DB USER	Utilisateur de la Base PostgreSQL Bronze
BRONZE DB PASSWORD	Mot de passe PostgreSQL Bronze
SILVER_DB_NAME	Nom de la BDD Silver
SILVER_DB_HOST	Host de la BDD Silver
SILVER_DB_USER	Utilisateur de la BDD Silver
SILVER_DB_PORT	Port de la BDD Silver
NEXT PUBLIC API URL	URL de l'API pour le frontend

Table 10: Variables d'Environnement Principales

18.3 C. D'épendances Python

Paquet	Version	Objectif
fastapi	0.115.5	Framework web pour construction d'APIs
uvicorn	0.32.1	Serveur ASGI pour FastAPI
psycopg2-binary	2.9.10	Adaptateur PostgreSQL pour Python
pydantic	2.10.3	Validation de données et gestion de paramètres
python-dotenv	1.0.1	Gestion de configuration d'environnement
python-multipart	0.0.20	Support pour données multipart
kafka-python	2.0+	Client Kafka (utilisé par extract/load)
requests	2.31+	Bibliothèque HTTP pour appels API

Table 11: D'épendances Python Principales

18.4 D. D'épendances Frontend

Les dépendances principales du frontend Next.js incluent :

- **next** : 16.0.3 - Framework React
- **react** : 19.2.0 - Bibliothèque UI
- **typescript** : 5.x - Typage statique
- **tailwindcss** : 4.1.9 - Framework CSS
- **recharts** : latest - Graphiques composables
- **@tanstack/react-query** : latest - Gestion d'état serveur
- **lucide-react** : Icônes React

- **radix-ui** : Composants UI accessibles